Estufa inteligente na plataforma FIWARE

Relatório técnico do Projeto 2

CCM520 - Internet das Coisas
Centro Universitário FEI

Júlia Bastos Barreto - RA 22124071

API key do grupo: juliaestufa2026

Novembro de 2026

1. Objetivo

O objetivo deste projeto é integrar a estufa inteligente do Projeto 1 à plataforma FIWARE. A estufa é uma ESP32 programada em MicroPython e simulada no Wokwi, que mede a temperatura e a umidade do ar com um DHT22, a umidade do solo e a luminosidade, e controla um relé de irrigação e um servomotor que abre a janela.

No Projeto 1, as medidas e os comandos passavam diretamente por tópicos MQTT no broker público da HiveMQ, e quem quisesse usar a estufa precisava conhecer esses tópicos. Neste projeto, a estufa passou a ser uma entidade de contexto no Orion Context Broker. Assim, qualquer aplicação consulta o estado atual e envia comandos pela API NGSI-v2, sem precisar saber nada de MQTT. A partir dessa base, foram montados o histórico das medidas, um alerta de solo seco e um painel no Grafana.

2. Arquitetura

A estufa e a plataforma ficam em redes diferentes, e o ponto de encontro entre elas é o broker MQTT público broker.hivemq.com, o mesmo do Projeto 1. A ESP32 publica as medidas nesse broker e recebe dele os comandos.

A plataforma roda no computador do grupo, em contêineres Docker definidos no docker-compose.yml. O Mosquitto é um broker MQTT local, na porta 1883, e mantém uma ponte com o HiveMQ que deixa passar apenas os três tópicos da API key do grupo. O IoT Agent JSON, nas portas 4041 e 7896, está ligado ao Mosquitto local e faz a tradução entre o JSON publicado pela estufa e o modelo NGSI-v2. O Orion, na porta 1026 e com o MongoDB, guarda o estado atual da estufa e recebe os comandos das aplicações.

Do lado de quem consome os dados, o Orion envia notificações para o Cygnus (porta 5050), que grava o histórico no MySQL (porta 3306), e para o consumidor (porta 3001 no computador), que recebe os alertas de solo seco. O Grafana (porta 3000) lê o histórico no MySQL e mostra o painel. O provisionamento e os comandos são feitos pelo Postman.

A estufa é a entidade urn:ngsi-ld:Greenhouse:001, do tipo Greenhouse, no serviço fei e caminho /estufa. O dispositivo se chama estufa001 e usa a API key juliaestufa2026. Para não haver conflito no broker público, a ESP32 usa o client id esp32_juliaestufa2026_estufa001 e a ponte usa ponte_juliaestufa2026. Os tópicos são /json/juliaestufa2026/estufa001/attrs para as medidas, /juliaestufa2026/estufa001/cmd para os comandos e /json/juliaestufa2026/estufa001/cmdexe para as respostas.

A estufa publica um único JSON com chaves curtas, que o provisionamento traduz para os nomes dos atributos:

Chave	Atributo	Unidade / faixa
t	temperature	°C
h	relativeHumidity	%
s	soilMoisture	% (0 seco, 100 encharcado)
l	illuminance	% (0 escuro, 100 claro)
r	relayState	0 ou 1
a	windowAngle	0 a 180
i	samplingInterval	2 a 10 s

Além das medidas, a entidade tem dois atributos estáticos, name e location, e quatro comandos: setRelay (0 ou 1), setServo (0 a 180), setInterval (2 a 10 s) e setSending (0 ou 1).

3. Decisões de projeto

A API key juliaestufa2026 foi usada igual nos três lugares em que ela aparece: na ponte, na coleção do Postman e no firmware. Se um deles tivesse um valor diferente, os tópicos não bateriam e as mensagens não chegariam. Para os object_id escolhemos uma letra para cada medida, o que deixa o JSON publicado pela ESP32 curto.

Os sensores de solo e de luz são lidos pelo ADC de 12 bits com atenuação de 11 dB. No circuito do Wokwi, um valor de ADC menor corresponde a solo mais úmido e a mais luz, então a conversão usa a leitura 4095 como 0 % e a leitura 0 como 100 %, limitando o resultado entre 0 e 100. Esses valores são os extremos da simulação e precisariam ser medidos de novo com os sensores reais.

O intervalo de coleta padrão é de 3 segundos, como pede o enunciado, e pode ser alterado pelo comando setInterval entre 2 e 10 segundos. Todo comando recebe uma resposta OK ou ERRO com o motivo, e um valor inválido nunca muda o estado da estufa. Mensagens com JSON inválido são ignoradas, para que nada publicado por engano no broker público derrube o programa. Depois de executar um comando, a estufa já publica uma medida, para que o Orion e o painel mostrem o novo estado sem esperar o próximo ciclo.

O laço principal não usa time.sleep() nem wait_msg(): ele verifica os comandos com check_msg() e controla o tempo das medidas com ticks_ms, de forma que medidas e comandos são atendidos ao mesmo tempo. O firmware também reconecta o Wi-Fi e o MQTT sozinho quando a conexão cai, assinando de novo o tópico de comandos (desafio D.2). O tempo limite da primeira conexão Wi-Fi foi aumentado para 30 segundos, porque o Wi-Fi simulado do Wokwi às vezes demorava mais de 15.

No provisionamento, o grupo de serviço foi criado com autoprovision desligado (ver R3). As subscrições usam os nomes dos serviços Docker, como cygnus:5050 e consumidor:3000, porque quem envia a notificação é o Orion, de dentro da rede Docker. O alerta de solo seco tem um throttling de 60 segundos: sem esse limite, cada nova medida geraria uma notificação enquanto o solo estivesse seco. O Grafana acessa o MySQL com o usuário grafana, que só tem permissão de leitura, já que o painel não precisa escrever no banco.

4. Testes e resultados

A coleção do Postman foi executada inteira no Runner, com a estufa ligada no Wokwi. A execução começa pela pasta "0 - Recomeçar do zero", que apaga o provisionamento e as subscrições, e as etapas seguintes recriam tudo. Foram 125 testes aprovados e nenhuma falha, como mostra o arquivo postman/runner.png. Esses testes verificam se os serviços estão no ar, o grupo e o dispositivo, a chegada das medidas ao Orion, os quatro comandos (inclusive valores inválidos, que devem ser recusados sem mudar o estado), o desligamento e a volta do envio, o histórico no MySQL, o alerta de solo seco e o painel do Grafana.

Como o Postman passou a exigir um plano pago para exportar o resultado do Runner, a mesma coleção também foi executada com o Newman, que é o executor de linha de comando do Postman. O resultado também foi de 125 testes aprovados, e o arquivo completo está em postman/Projeto 2.newman-resultados.json.

5. Limitações e dificuldades

Em duas ocasiões, depois de o Docker ficar alguns instantes sem responder, a ponte do Mosquitto reconectou ao HiveMQ mas parou de trazer as mensagens. As medidas continuavam chegando ao broker público, mas não ao Orion, e o problema só foi resolvido reiniciando o Mosquitto com docker compose restart mosquitto.

Outra dificuldade foi a velocidade do Wokwi. Quando a aba do simulador ficava em segundo plano, o navegador desacelerava a simulação, e os testes que dependem de tempo, como o 3.15 e o 4.2, falhavam. Com a janela do Wokwi visível, a coleção passou inteira.

Por fim, a calibração dos sensores foi feita só na simulação, e o broker público não tem autenticação nem controle de acesso, o que o torna adequado apenas para dados de laboratório (ver R8).

6. Uso de ferramentas de IA

Foi usado um assistente de IA para: diagnosticar problemas do ambiente (virtualização desligada na BIOS, limite de downloads do Docker Hub, ponte do Mosquitto travada e lentidão

do Wokwi); revisar os corpos JSON da coleção feitos por mim; e gerar o resultado da coleção com o Newman, já que a opção pedida exigia uma assinatura paga. O assistente também me guiou na escrita das seções 1 a 5 deste relatório. As respostas R1 a R8 foram escritas por mim, com revisão do assistente.

7. Respostas às perguntas R1 a R8

R1. O que é uma assinatura compartilhada (shared subscription)? O que aconteceria com a estufa, em uma turma de 20 grupos, se o IoT Agent de cada grupo se conectasse direto ao broker público? Como a ponte filtrada pela API key evita o problema e por que cada tópico da ponte tem a direção que tem?

Na assinatura compartilhada, os clientes que existem assinam como grupo e as mensagens publicadas nesse grupo são divididas pela quantidade de clientes que existem. Então o HiveMQ dividiria as mensagens entre todas as pessoas, já que o $share/json é o mesmo para todos e o IoT Agent assina com curinga e pega mensagens de qualquer API key; e todas as medidas da estufa, que é sua propriedade, também seriam compartilhadas, ou seja, as outras pessoas descartariam as mensagens, já que não são delas e não são úteis para elas. Para resolver isso, é só ter o seu Mosquitto local no PC, onde ele receberia tudo, já que é local e seria o "único", e a ponte só traria os tópicos com a sua API key, sem necessidade da API key dos outros clientes. A direção depende de quem publica a mensagem: as medidas (attrs) e as respostas (cmdexe) são publicadas pela ESP32 no HiveMQ, por isso são in; os comandos (cmd) são publicados pelo IoT Agent no PC, por isso são out. Se todas fossem para os dois sentidos, poderia acontecer de gerar cópias.

R2. Por que a API key precisa ser única na turma? O que aconteceria se dois grupos usassem a mesma API key e o mesmo device_id?

A API key precisa ser única justamente para que todo mundo receba somente as informações da sua respectiva estufa. Se todo mundo usasse a mesma API key, as estufas ficariam no mesmo endereço e as medidas dos grupos se misturariam, aí ninguém saberia qual medida é de quem. Por exemplo, se alguém desse setRelay = 1, todas as estufas executariam, e a consequência seria um grupo ligar ou desligar o relé da estufa do outro sem querer.

R3. Por que o grupo deve rejeitar dispositivos não provisionados no broker público? Dê um exemplo do que poderia acontecer sem isso.

Deve rejeitar porque, no broker público, qualquer pessoa pode publicar nos tópicos com a nossa API key. Somente o dispositivo cadastrado no IoT Agent, com o mesmo device_id, deve ser aceito; caso contrário, é rejeitado, já que o device_id é basicamente a identificação da estufa. Se chegar outro device_id, quer dizer que não é a nossa estufa. Se o IoT Agent aceitasse qualquer device_id, alguém poderia criar uma estufa falsa, com dados falsos.

R4. Descreva o caminho de uma medida, da ESP32 até o painel do Grafana: componente, protocolo, porta e formato do dado em cada salto.

A ESP32 mede a temperatura e manda pela rede usando o protocolo MQTT, na porta 1883, em um JSON com chaves curtas, como {"t": 24.0}. O HiveMQ recebe a mensagem, e a ponte traz para o Mosquitto, no meu PC, também por MQTT na porta 1883, sem mudar o JSON. Depois, o Mosquitto entrega para o IoT Agent, do mesmo jeito. O IoT Agent traduz as chaves curtas usando os object_id, por exemplo t vira temperature, e manda para o Orion usando HTTP (NGSI-v2), na porta 1026. O Orion guarda o valor atual. Quando chega uma medida nova, o Orion avisa o Cygnus por HTTP, na porta 5050, no formato normalized. O Cygnus grava cada medida como uma linha no MySQL, na porta 3306. Por fim, o Grafana consulta o MySQL e desenha o gráfico, que eu vejo no navegador na porta 3000.

R5. Descreva o caminho de um setRelay, do Postman até o relé e de volta. Quando o _status fica PENDING e quando vira OK? Por que OK não garante que a estufa aceitou o comando (requisição 3.7)?

Na ida, o Postman manda o setRelay = 1 para o Orion. Como é o IoT Agent que gerencia os comandos, o Orion repassa para ele, e o IoT Agent publica esse comando no MQTT (no Mosquitto) e fica esperando a resposta, então o _status fica PENDING. A ponte do Mosquitto leva o comando para o HiveMQ, e a ESP32 recebe e liga o relé. Na volta, a ESP32 responde {"setRelay": "OK"} no HiveMQ, a ponte traz a resposta para o Mosquitto, e o IoT Agent avisa o Orion. Aí o _status vira OK. Mas o _status = OK só quer dizer que a estufa respondeu. Se ela aceitou ou não, quem diz é o _info: na requisição 3.7, por exemplo, a janela a 200° deu _status = OK, mas o _info foi ERRO.

R6. Por que setSending = 0 não pode desligar a escuta de comandos?

Com setSending = 0, só as medidas param, os comandos não. As medidas são feitas para eu ter informações da estufa, e a qualquer momento posso solicitar que voltem. Já os comandos não podem parar, porque são essenciais para o funcionamento da estufa. Se a estufa parasse de escutar os comandos, não teria como mandar o setSending = 1 para religar o envio, e ela ficaria sem mandar medidas até alguém reiniciar a ESP32. Também não teria como ligar o relé para irrigar, por exemplo. Por isso só as medidas param e os comandos não.

R7. O que o Orion guarda e o que o MySQL guarda? Por que a subscrição do Cygnus usa normalized e a do alerta usa keyValues?

O Orion guarda somente as medidas atuais da estufa, e o MySQL é um banco de dados que guarda o histórico dos valores que vão mudando no Orion. O Cygnus usa o normalized porque é o que vai ser gravado no banco de dados, então ele precisa da estrutura completa de cada atributo, com o valor e o tipo, e o Cygnus só aceita esse formato. Já o alerta usa keyValues porque é algo mais simples, feito somente para o consumidor observar o valor.

R8. Qualquer pessoa conectada ao broker público pode ler e publicar nos seus tópicos. Descreva dois ataques possíveis à estufa e duas medidas que os evitariam em um sistema real.

Um ataque possível é alguém publicar setRelay = 1 no meu tópico de comandos. Pode ocorrer de alagar a estufa, porque a ESP32 vai simplesmente obedecer, já que qualquer um pode ter acesso ao broker público e não dá para saber quem mandou. Outro ataque é alguém derrubar a conexão da estufa: como o broker é público, a pessoa tem acesso à minha API key e ao meu device_id, monta o mesmo client id da ESP32 e derruba a minha sessão para entrar na dela. Para evitar isso, num sistema real, poderia ser usado um broker privado que precisa de autenticação, que dá para identificar os usuários, e assim só entra quem tem permissão.

8. Referências

Material das Aulas 5 a 9 da disciplina CCM520 e guia passo a passo do Projeto 2.
FIWARE IoT Agent JSON: https://fiware-iotagent-json.readthedocs.io
FIWARE Orion (NGSI-v2): https://fiware-orion.readthedocs.io
HiveMQ MQTT Essentials: https://www.hivemq.com/mqtt/